What are standard port no associated with following :

1. HTTP: 80
2. HTTPS: 443
3. FTP: 21
4. SMTP: 25
5. SSH: 22

Ports are a dedicated way of sharing information, there are 1000+ ports no where some of them are fixed like above.

Validations Overview , Validations with Data Annotation , Server Side and Client Side Validation , Custom Server side validation , Model level validation using IValidatableObject , Custom unobstrive Client side Validation, Remote Validation, Url Routing Overview, Custom Routes, Attribute Routing , Routing Constraints

Cookies , Sessions, MVC Filters,Customizing Filters,Configuring Filters and logging in ASP.NEt Core

What are different ways of implementing validation in ASP.NET Core Web Application ?

When to use client side and server side validations in ASP.NET Core MVC Web Application?

Steps for implementing routing and its different types ?

Validation Type	Purpose	Common Usage
Data Annotation Validation	Built-in attribute-based validation	Simple form validations
Client-Side Validation	Validation in browser using JavaScript/jQuery	Better user experience
Server-Side Validation	Validation on server after	Security and data integrity

	form submission	
Custom Validation	Business-rule-based validation	Domain-specific logic
Model-Level Validation (IValidatableObject)	Cross-property validation	Multiple field dependency
Remote Validation	AJAX-based validation from server	Username/email uniqueness
Custom Unobtrusive Validation	Advanced custom client rules	Enterprise-grade forms

Server side validation

```
using
System.ComponentModel.DataAnnotations;

public class CustomAgeValidation :
ValidationAttribute
{
    protected override ValidationResult
IsValid(object value,
        ValidationContext
validationContext)
    {
        int age = (int)value;

        if(age < 18)
        {
            return new
ValidationResult("Age must be above 18");
        }

        return ValidationResult.Success;
    }
```

```
[CustomAgeValidation]
public int Age { get; set; }
```

}	

MVC project

	using System.Collections.Generic;
	using
	System.ComponentModel.DataAnnotations;
	
	public class Student : IValidatableObject
	{
	public string Name { get; set; }
	public int Age { get; set; }
	
	public IEnumerable<ValidationResult>
	Validate(
	ValidationContext
	validationContext)
	{
	if(Name == "Admin" && Age < 25)
	{
	yield return new
	ValidationResult(
	"Admin must be above
	25");
	}
	}
	}

Routing Type	Description
Conventional Routing	Centralized route configuration
Attribute Routing	Route defined on controller/action

Custom Routing	User-defined route patterns
Routing Constraints	Restrict route parameters

Case Study: Online Food Delivery Portal – Routing in ASP.NET Core MVC

Scenario Overview

A food delivery company is developing a web application using ASP.NET Core.

The application should:

- Display restaurant menus
- Navigate using clean URLs
- Restrict invalid routes
- Support SEO-friendly routing

The development team decides to implement:

- URL Routing
- Custom Routes
- Attribute Routing
- Routing Constraints

User Story ID	User Story
US-101	As a customer, I want clean URLs so that navigation becomes easier

US-102	As an admin, I want custom routes for restaurant modules
US-103	As a developer, I want attribute routing for better API management
US-104	As a system, I want route constraints to prevent invalid URLs

Topic	Business Requirement	Features	Step-by-Step Implementation	Example URL	Best Practices
URL Routing Overview	Navigate between restaurant pages	Maps URL to Controller & Action	1. Configure routing in Program.cs 2. Define default route3. Create controller and action	/Restaurant/Menu	Use meaningful URLsKeep route structure simple
Custom Routes	Create user-friendly URLs for food ordering	SEO-friendly routes	1. Add custom route in Program.cs 2. Define controller/action mapping	/order-food	Use readable namesAvoid overly complex patterns
Attribute Routing	Better control over API	Route defined directly on controller/act	1. Use [Route()] attribute	/restaurant/details	Preferred for APIsMaintain

	endpoints	ion	2. Apply route at controller/action level		consistency
Routing Constraints	Prevent invalid route values	Restricts route parameters	1. Define route constraint 2. Restrict datatype (int, alpha)	/restaurant/101	Use constraints for validation and security

Steps for implementing above demo in ASP.NET MVC application :

Step 1: In FoodDeliveryApp we will start with creating a controller first ie **Controllers/RestaurantController.cs**

```
using Microsoft.AspNetCore.Mvc;

public class RestaurantController : Controller
{
    public IActionResult Menu()
    {
        return Content("Restaurant Menu Page");
    }

    public IActionResult Details(int id)
    {
        return Content("Restaurant Details ID: " + id);
    }
}
```

Step 2: Implementing Default routing via program.cs(program execution starts from here)

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

if we want this **/Restaurant/Menu and /order-food**

```
app.MapControllerRoute(
```

```
    name: "foodOrder",
    pattern: "order-food",
    defaults: new
    {
        controller = "Restaurant",
        action = "Menu"
    });
```

Step 3: Implement Attribute Routing

```
[Route("restaurant")]
public class RestaurantController : Controller
{
    [Route("menu")]
    public IActionResult Menu()
    {
        return Content("Restaurant Menu");
    }

    [Route("details/{id}")] // this how attribute routing is implemented
    public IActionResult Details(int id)
    {
        return Content("Restaurant Details: " + id);
    }
}
```

to invoke above action methods user can :
/restaurant/menu

/restaurant/details/5
Step 5: Adding Routing constraints:

```
[Route("details/{id:int}")]
public IActionResult Details(int id)
{
    return Content("Restaurant ID: " + id);
}
```

now the valid URL will be **/restaurant/details/10 for invoking above action method**

Best Practices

- Use meaningful route names
- Prefer attribute routing for APIs

- Apply route constraints wherever possible
- Keep URLs short and readable
- Maintain consistent naming conventions

Case Study: Online Examination Portal – Session Management, Cookies, Filters & Logging in ASP.NET Core MVC

Scenario Overview

A university is developing an online examination portal using ASP.NET Core.

The application should:

- Maintain student login sessions
- Remember user preferences using cookies
- Restrict unauthorized access
- Track user activities through logging
- Apply reusable validation and security logic using filters

The development team decides to implement:

- Cookies
- Sessions
- MVC Filters
- Custom Filters
- Filter Configuration
- Logging Mechanism

User Stories

User Story ID	User Story
US-201	As a student, I want the system to remember my login session

US-202	As a user, I want my preferred theme/language saved
US-203	As an admin, I want unauthorized users blocked
US-204	As a developer, I want centralized logging for troubleshooting
US-205	As a system, I want reusable filters for security and monitoring

Topic	Business Requirement	Features	Step-by-Step Implementation	Example	Best Practices
Cookies	Store lightweight user preferences	Small browser-based storage	1. Create cookie2. Store user preference3. Read cookie	Save selected theme	Avoid storing sensitive data
Sessions	Maintain user login information	Server-side user state management	1. Configure session2. Store session data3. Retrieve session	Logged-in student details	Clear session after logout
MVC Filters	Execute reusable logic before/after requests	Cross-cutting concerns handling	1. Create filter2. Apply on controller/action	Logging filter	Keep filters lightweight

Custom Filters	Implement custom security/logging	Reusable validation and monitoring	1. Create custom filter class2. Override methods	Access validation	Use dependency injection
Configuring Filters	Apply filters globally/controller/action level	Centralized configuration	1. Register filters in Program.cs	Global logging	Avoid unnecessary global filters
Logging	Track errors and user activity	Monitoring & debugging	1. Inject logger2. Write logs	Login tracking	Use structured logging

Step-by-Step Demo Implementation

Step 1: Create ASP.NET Core MVC Project

1. Open Visual Studio
2. Create → ASP.NET Core Web App (MVC)
3. Project Name:

```
None
OnlineExamPortal
```

Step 2: Configure Session

Program.cs

```
None
builder.Services.AddSession();


builder.Services.AddControllersWithViews();


var app = builder.Build();


app.UseSession();
```

Why?

Sessions help maintain user login state because HTTP is stateless.

Step 3: Store Session After Login

Controllers/AccountController.cs

```
None
using Microsoft.AspNetCore.Mvc;


public class AccountController : Controller
{
   public IActionResult Login()
   {
       return View();
```

```
    }


    [HttpPost]
    public IActionResult Login(string username)
    {
        HttpContext.Session.SetString("User", username);


        return RedirectToAction("Dashboard");
    }


    public IActionResult Dashboard()
    {
        var user = HttpContext.Session.GetString("User");


        return Content("Welcome " + user);
    }
}
```

User Story Mapping

User Story	Implementation

Student wants login persistence	Session storage

Step 4: Implement Cookies

Store Cookie

```
None

Response.Cookies.Append("Theme", "Dark");
```

Read Cookie

```
None

var theme = Request.Cookies["Theme"];
```

User Story Mapping

User Story	Implementation
User preference storage	Cookies

Step 5: Create Custom Logging Filter

Filters/CustomLogFilter.cs

None

```
using Microsoft.AspNetCore.Mvc.Filters;


public class CustomLogFilter : ActionFilterAttribute
{
   public override void OnActionExecuting(
       ActionExecutingContext context)
   {
       Console.WriteLine("Action Executing");
   }


   public override void OnActionExecuted(
       ActionExecutedContext context)
   {
       Console.WriteLine("Action Executed");
   }
}
```

Features of Filters

Feature	Purpose
Reusable Logic	Avoid duplicate code

Logging	Monitor application flow
Security	Restrict unauthorized access
Exception Handling	Centralized error management

Step 6: Apply Filter on Controller

```
None
[CustomLogFilter]
public class ExamController : Controller
{
   public IActionResult StartExam()
   {
       return View();
   }
}
```

User Story Mapping

User Story	Implementation
Admin wants activity monitoring	Logging filter

Step 7: Configure Global Filters

Program.cs

```
None
builder.Services.AddControllersWithViews(options =>
{
   options.Filters.Add(typeof(CustomLogFilter));
});
```

Why Global Filters?

Applies common functionality across entire application.

Step 8: Implement Logging

Controller Logging

```
None
private readonly ILogger<HomeController> _logger;


public HomeController(ILogger<HomeController> logger)
{
   _logger = logger;
}
```

```
public IActionResult Index()
{
    _logger.LogInformation("Home page accessed");

    return View();
}
```

Logging Benefits

Benefit	Description
Debugging	Easier troubleshooting
Monitoring	Track user activity
Auditing	Security compliance
Error Tracking	Capture application failures

Difference Between Cookies and Sessions

Feature	Cookies	Sessions

Storage	Browser	Server
Security	Less secure	More secure
Lifetime	Persistent/temporary	Session-based
Usage	Preferences	Login/user state

Types of MVC Filters

Filter Type	Purpose
Authorization Filter	Authentication & access control
Action Filter	Before/after controller action
Result Filter	Before/after result execution
Exception Filter	Handle exceptions
Resource Filter	Optimize request pipeline

Common Mistakes

Mistake	Impact
Not clearing sessions	Security risks
Storing sensitive data in cookies	Data exposure
Heavy logic inside filters	Performance issues
No logging strategy	Difficult debugging

Best Practices

- Use sessions for authentication-related data
- Use cookies only for lightweight preferences
- Keep filters reusable and modular
- Use centralized logging strategy
- Avoid sensitive data inside cookies
- Clear session on logout